



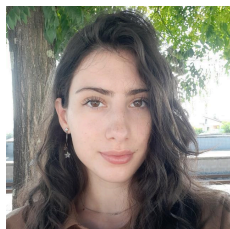
Universidade do Minho

Mestrado em Engenharia Informática

Unidade Curricular de Engenharia de Serviços em Rede

Ano Lectivo de 2024/2025

Grupo 85



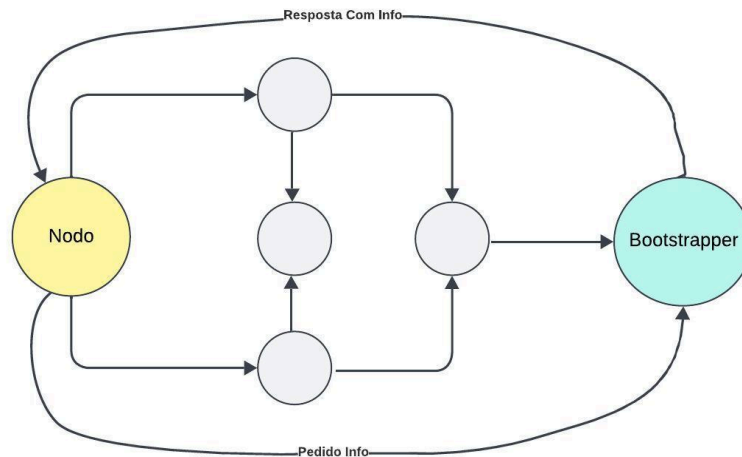
Mariana Antunes Silva (PG55980)



João Paulo Campelo Gomes (PG55960)

Interações

1. Método para conhecimento dos vizinhos



O bootstrapper tem como função assegurar que um nó recém-iniciado consiga integrar-se na rede, o bootstrapper tem acesso a dois ficheiros JSON: um contendo a informação relativa aos vizinhos e o respectivo identificador IP dele próprio na rede, e outro que contém os endereços IP de todos os servidores. Estes ficheiros são fornecidos como argumentos no momento da execução e são posteriormente processados, sendo a informação armazenada numa tabela.

Existem dois tipos de comunicação entre o bootstrapper e os restantes nodos:

- **Comunicação entre bootstrapper e pontos de acesso**

Quando um ponto de acesso (PA) se conectar à rede, é necessário que este conheça os seus vizinhos, o seu identificador IP e os servidores disponíveis. Este processo é iniciado por um pedido enviado pelos PA ou NI, no qual indicam, através do conteúdo da mensagem, que desejam obter informações do bootstrapper, incluindo um código GET_INFO (2). Ao receber essa mensagem, o bootstrapper reconhece o tipo de resposta que deve enviar. Nesse caso, a resposta terá o conteúdo semelhante a: {"vizinhos": lista de vizinhos, "ipIdentificador": identificador IP, "servidores": servidores disponíveis na rede}. O nodo que fez a solicitação recebe essa mensagem e armazena as informações recebidas nos locais apropriados.

- **Comunicação entre bootstrapper e nodos intermédio**

Quando um nodo intermediário (NI) se conecta à rede, é necessário que este conheça os seus vizinhos, o seu identificador IP, os servidores e os pontos de acesso disponíveis. Os NI precisam de conhecer os pontos de acesso para poderem conhecer os destinos possíveis para poder estabelecer rota. Este processo é idêntico ao descrito em cima apenas difere no

conteúdo da mensagem que envia que vai com um código GET_INFO (3) e na mensagem que recebe contendo esta {"vizinhos": lista de vizinhos, "ipIdentificador": identificador IP, "servidores": servidores disponíveis na rede, "acess_point": lista de pontos de acesso}.

- **Comunicação entre bootstrapper e clientes e servidores**

O processo é similar ao descrito anteriormente, com duas diferenças principais. A primeira é que o código inserido na mensagem enviada ao bootstrapper, por parte do cliente ou servidor, é o código GET_INFO (1). A segunda diferença está na resposta recebida, que contém apenas a seguinte informação: {"vizinhos": lista de vizinhos, "ipIdentificador": identificador IP}. Isso ocorre porque nem os clientes nem os servidores necessitam de conhecer os servidores da rede.

2. Ligação Cliente - Ponto de acesso

Antes de haver qualquer pedido entre cliente - ponto de acesso o cliente irá avaliar os tempos de resposta entre os pontos de acesso aos quais tem ligação depois de verificar qual o ponto de acesso que responde mais depressa este será o selecionado para comunicação.

3. Método para conhecimento dos vídeos disponíveis na rede

Ao se ligarem à rede os pontos de acesso (PA) e os nodos intermédios (NI) recebem do bootstrapper informação acerca do servidor da rede nomeadamente o ip deste, sendo assim os PA e NI manda um pedido ao servidor para a porta (CHECK_VIDEOS_PORT = 3001) a pedir para o servidor lhe mandar os vídeos que este tem, depois de receber a mensagem de resposta por parte do servidor a informação é guardada numa tabela que contém o ip do servidor e a lista de vídeos que ele tem ({"ipServer": [vídeos]}).

4. Pedido de verificação de um vídeo

No processo de verificação da disponibilidade de um vídeo em uma rede distribuída, um cliente precisa determinar se o vídeo desejado está disponível em algum ponto de acesso (nó) da rede. A seguir, detalha-se a sequência de operações realizadas durante a interação entre o cliente e o ponto de acesso para verificar a existência do vídeo.

- **Pedido de Verificação de Existência do Vídeo**

Quando o cliente deseja visualizar um vídeo, o primeiro passo consiste em verificar se o vídeo está disponível na rede. Para isso, o cliente envia uma mensagem de verificação ao seu vizinho mais próximo (ponto de acesso), utilizando um socket UDP. O cliente cria uma mensagem que contém o nome do vídeo (conteudo) e o seu próprio identificador de IP. A mensagem é enviada através de um socket para o endereço e porta do vizinho

(CHECK_VIDEOS_PORT = 3001), com um número máximo de tentativas em caso de falha na comunicação.

- **Resposta do Ponto de Acesso**

O ponto de acesso que recebe o pedido de verificação analisa a mensagem e verifica se o vídeo solicitado está disponível em sua tabela interna. Em seguida, o ponto de acesso envia uma resposta ao cliente, indicando se o vídeo existe ou não. Se o vídeo for encontrado no ponto de acesso, é enviada uma mensagem afirmativa (True), indicando que o vídeo está disponível. Caso contrário, é enviada uma mensagem negativa (False), informando que o vídeo não existe na rede.

- **Processamento da Resposta pelo Cliente**

Após enviar o pedido, o cliente aguarda a resposta do ponto de acesso. Se o vídeo estiver disponível, o cliente prossegue com o pedido de transmissão do vídeo. Caso contrário, o cliente recebe uma notificação de que o vídeo não existe na rede.

5. Stream e protocolos

A escolha do protocolo aplicacional para o streaming recaiu sobre o RTP (Real-time Transport Protocol). Este protocolo possibilita o envio de mensagens através do protocolo de transporte UDP, que se destaca pela sua simplicidade e baixa latência, características essenciais para transmissões em tempo real. Adicionalmente, o RTP é particularmente adequado para cenários que apresentam tolerância a perdas de pacotes, oferecendo, assim, uma solução eficaz e compatível com a natureza do nosso projeto.

A transmissão de um vídeo através do RP consiste na recepção dos frames vindos do servidor que posteriormente são reencaminhados para os nós que fazem parte do trajeto do vídeo.

A implementação da transmissão de dados foi realizada com:

- No servidor, para cada vídeo em transmissão, é implementada uma lógica específica para realizar a transmissão do conteúdo como pacotes RTP (Real-time Transport Protocol) sobre o protocolo UDP. Essa lógica é responsável pela gestão da comunicação com os nós que solicitam a reprodução do vídeo. O servidor controla o envio dos pacotes RTP contendo os dados do vídeo, coordenando de forma eficiente a leitura e o envio contínuo dos frames de vídeo para os destinos apropriados.
- A classe **RtpPacket** é utilizada pelo servidor e é responsável pela criação, codificação e decodificação de pacotes RTP, um protocolo utilizado para a transmissão de dados de áudio e vídeo em tempo real sobre redes IP. Especificamente, esta classe é utilizada para empacotar os dados de vídeo, como os frames, e enviá-los de forma eficiente e organizada. O RTP encapsula os dados de vídeo permitindo mesmo que caso de perdas ou atrasos a stream “salte” esses frames mantendo a stream sincronizada entre clientes e evita a visualização de frames passados.
- A classe **VideoStream** é utilizada pelo servidor e é encarregada de ler e fornecer os frames de vídeo a partir de um arquivo, preparando-os para transmissão. Esta classe

simula a leitura de um ficheiro de vídeo e o empacotamento dos frames em pacotes RTP para serem enviados pela rede. Ela assegura que os frames sejam extraídos sequencialmente e prontos para serem empacotados e transmitidos ao cliente.

- A classe **ClienteStream** gere a interface gráfica (GUI) e a comunicação de rede de um cliente que recebe e exibe vídeo via RTP. Esta classe é responsável pelo controlo da reprodução dos vídeos recebidos, incluindo a gestão de botões de controlo (como Play, Pause e Teardown), processamento dos pacotes RTP e atualização da interface de utilizador com os frames de vídeo recebidos. O ClienteStream garante que o cliente consiga visualizar corretamente o vídeo transmitido, com a capacidade de interagir com a reprodução de acordo com os comandos do utilizador.

Essas classes trabalham em conjunto para possibilitar a transmissão de vídeo em tempo real utilizando o protocolo RTP, garantindo a comunicação eficiente entre o servidor e o cliente, bem como a reprodução contínua dos dados de vídeo.

6. Construção da árvore

Os principais objetivos deste trabalho consistem em desenvolver uma implementação de multicast que minimize a carga na rede sem comprometer a latência e, idealmente, com suporte a tolerância a falhas. Para atingir esses objetivos, a nossa rede overlay foi concebida com uma gestão descentralizada, adotando o princípio de que nenhum dos nós intermédios é completamente confiável, pois estes podem falhar ou apresentar deterioração das condições a qualquer momento.

Para suportar esta abordagem, cada nó na rede overlay possui uma tabela que armazena os endereços IP dos pontos de acesso, associando a cada um destes um dicionário ordenado. Este dicionário utiliza como chave o IP de um nó vizinho e como valor um conjunto de métricas, incluindo:

- Melhor tempo de resposta;
- Número de rotas adicionais disponíveis;
- Número de trocas de mensagens bem-sucedidas;
- Número de falhas (vezes em que o nó esteve inoperacional);
- Pontuação.

O dicionário é ordenado com base na pontuação, calculada através do somatório das métricas ponderadas por fatores específicos (valores negativos são atribuídos a parâmetros que indicam falhas).

A estrutura adotada permite que as tabelas considerem apenas o destino final, sem a necessidade de identificar a origem da solicitação do vídeo. Este sistema possibilita a recuperação de falhas através de uma simples troca de mensagens ou timeout, evitando a reconstrução completa da árvore de encaminhamento.

O cálculo do melhor tempo de cada nó é realizado somando o melhor tempo dos seus vizinhos ao *trip time* respetivo. Além disso, os pontos de acesso também enviam métricas aos seus nós vizinhos, simulando o comportamento de nós comuns. Essa abordagem permite a construção da árvore de encaminhamento com o ponto de acesso como raiz, uma vez que este terá sempre o melhor tempo para si próprio (zero).

Caso um nodo vizinho falhe em enviar novas métricas dentro de um tempo pré-definido, este é considerado como inativo e é colocado como tendo como melhor tempo infinito e incrementa-se o número de falhas em 1 de forma a colocá-lo no fundo da tabela e de baixar a sua pontuação no futuro quando este se reconectar.

Na nossa implementação, o tempo pelo qual ele está embaixo não interfere na penalização, pois só é incrementado o número de falhas, caso o nodo fosse considerado “vivo” quando se detetou a falha deste a enviar métricas.

7. Transmissão de vídeo

Para realizar a transmissão de um vídeo, utilizamos a camada de rede para efetuar os pedidos de transmissão, uma vez que, sendo pedidos únicos, não há vantagens em recorrer à nossa rede overlay para este propósito. Quando um pedido é enviado ao servidor, este adiciona o ponto de acesso à lista de destinos do vídeo. Se for o primeiro pedido para o vídeo em questão, o servidor inicia a transmissão (*stream*); caso contrário, o novo destino é incluído no cabeçalho dos pacotes subsequentes da *stream*.

O servidor encaminha os pacotes para a sua conexão com o overlay, utilizando a porta **3004 + ID do vídeo**, em que o ID do vídeo é um número inteiro positivo único para cada vídeo.

Quando um nó da rede overlay recebe um pacote, procede da seguinte forma:

1. Analisa os destinos indicados no pacote.
2. Determina a melhor rota para cada destino.
3. Caso os destinos partilhem a mesma rota, o nó mantém o pacote inalterado e envia-o para o próximo nó calculado como o mais adequado.
4. Se houver destinos com rotas diferentes, o nó ajusta os pacotes de forma a agrupar os destinos que compartilham o mesmo próximo nó, enviando pacotes separados para cada grupo de destinos. Por exemplo, se existem três destinos (destino1, destino2 e destino3) e, após o cálculo de rotas, destino1 e destino2 partilham o mesmo próximo nó, o pacote será ajustado para incluir apenas esses dois destinos e será enviado como um único pacote para o nó correspondente. Em seguida, o nó cria outro pacote contendo exclusivamente destino3 e envia-o para o próximo nó definido como melhor rota para esse destino.

Este processo é repetido até que os pacotes atinjam os pontos de acesso indicados como destinos.

Para prevenir loops, o nó selecionado como a melhor rota nunca pode ser o mesmo de onde o pacote foi recebido. Esta regra também se aplica à transmissão de métricas entre nós vizinhos.

8. Paragem da transmissão de um vídeo

Se um cliente pretender parar de assistir o stream, este irá enviar uma mensagem ao ponto de acesso com o nome do vídeo que pretende parar, o ponto de acesso irá receber esta mensagem pela porta (`STOP_VIDEOS_PORT` = 3003) e irá remover o cliente que enviou o pedido de encerramento da sua lista de interessados. Se o ponto de acesso verificar que a sua lista de interessados é vazia, para aquele vídeo, este comunica ao servidor que já não tem interesse num determinado vídeo, fazendo com que o servidor remova o ponto de acesso da sua lista de destinatários para aquele vídeo e caso esta fique vazia a transmissão é parada.

9. Problemas e possíveis melhorias

Apesar de termos implementado com sucesso um sistema de streaming com multicast e tolerância de falhas, a nossa implementação precisaria de alguns ajustes para ser utilizada num caso real. Nós temos consciência destes problemas, mas devido a limites de tempo e de que tivemos de realizar o trabalho como um par, não nos foi possível implementar tudo o que tínhamos planeado, pelo que passamos a expor aqui a lista de problemas, potenciais soluções e melhorias da nossa implementação:

→ Problema 1: Assumimos que os pontos de acesso estão sempre vivos e não permitimos que o cliente troque de ponto de acesso

Num cenário real, seria necessário implementar um mecanismo que permitisse ao cliente avaliar periodicamente outros pontos de acesso disponíveis. Dessa forma, o cliente poderia cancelar a sua ligação com o ponto de acesso original e migrar para um novo ponto de acesso que oferecesse melhor qualidade de serviço (QoS).

Para que esta transição seja indetetável para o utilizador, o cliente só enviaria o pedido de cancelamento ao PA original após começar a receber a transmissão do novo PA. Este procedimento evita interrupções no fluxo de dados, garantindo que a transmissão continua ininterrupta. Caso o cancelamento fosse feito imediatamente, o cliente ficaria sem receber a *stream* até que o novo PA completasse o registo do cliente ou, no pior cenário, até que o novo PA obtivesse o vídeo em questão.

→ Problema 2: Estamos a usar apenas o port para identificar o vídeo

Como nós temos um número bastante reduzido de vídeos utilizar `3004 + id do vídeo`, nunca irá passar do número máximo de port, no entanto se a nossa aplicação tivesse a dimensão do twitch seria impossível mantermos este sistema, pois em pouco tempo passaríamos do número limite. Para resolver este problema, há uma série de implementações diferentes, mas nenhuma é perfeita, uma das implementações passaria por reutilizar ids quando as streams terminassem, mas esta solução também não funcionaria num sistema de grande dimensão, pois se houvessem mais streams em simultâneo do que números de portas o sistema iria falhar.

Outra solução possível seria utilizar o resto da divisão inteira do número limite de ports pelo id do vídeo e utilizar esse resto juntamente com o id do vídeo no cabeçalho do pacote para determinar que vídeo é. Esta solução já nos permite ter qualquer número de streams em simultâneo, mas adiciona informação ao cabeçalho e necessita de computação adicional o que causaria uma diminuição na velocidade de transmissão comparativamente com as outras.

→ Problema 3: Estamos a usar um cabeçalho no envio dos pacotes de vídeo

Para garantirmos o comportamento que nos permite recuperar de falhas com uma única troca de mensagens, precisamos de adicionar um pequeno cabeçalho aos pacotes de vídeo, o que leva a que a transmissão destes seja ligeiramente mais lenta do que se não o tivesse. Pelo que a nossa implementação seria pior do que se tivéssemos implementado uma versão onde cada nodo guarda o nodo que lhe pediu o vídeo numa rede onde falhas são muito incomuns. Isto não é necessariamente um problema de implementação, pois há vantagens e desvantagens sobre a outra, mas é algo que ponderamos quando implementamos esta versão.

→ Problema 4: Não aceitamos entrada dinâmica de nodos e pontos de acesso

No nosso sistema, todas as entidades comunicam com os seus vizinhos para diversas finalidades, e os nós da rede overlay possuem tolerância a falhas, sendo capazes de restabelecer comunicação quando os nós vizinhos que estavam inativos retomam a sua atividade. Contudo, a comunicação entre entidades é restrita às conexões previamente definidas pelo *Bootstrapper* (que poderia ser, por exemplo, um ficheiro de configuração).

Esta abordagem apresenta uma limitação importante em cenários reais, onde podem ser lançadas novas máquinas e os endereços IP das entidades podem ser dinâmicos. Nessas condições, a nossa implementação seria inviável, uma vez que não permite a deteção automática de novas entidades ou a adaptação a alterações nos endereços IP. Para resolver este problema, seria necessária uma arquitetura muito diferente da que temos, pois teríamos que utilizar algo como um DDNS (Dynamic Domain Name System) e um Bootstrapper dinâmico que informaria os nodos na região de uma nova máquina que essa máquina existia.

Referências:

<https://youtu.be/OxAvxsL08Cc?si=rVorlrmXFfU9WSe4>

